



Q1 • 2022

ThinkstScapes Quarterly



Brought to you by



**Most companies find out way too late
that they've been breached.**

Thinkst Canary changes this.

Canaries deploy in under 4 minutes
and require 0 ongoing admin overhead.

They remain silent until they need to chirp,
and then, you receive that single alert.

When.it.matters.

Find out why some of the smartest security teams
in the world swear by Thinkst Canary.

<https://canary.love>

A curious giraffe at Kruger National Park, South Africa.

Contents

Introduction	4
Themes covered in this issue	5
Low-level, but high-privilege bug hunting	6
Hyntrospect: a fuzzer for Hyper-V devices	7
Put an io_uring on it: Exploiting the Linux Kernel	8
The AMD Branch (Mis)predictor: Where No CPU has Gone Before	9
Dynamic Process Isolation	10
Another Brick in the Wall: Uncovering SMM Vulnerabilities in HP Firmware	11
Confidential computing for the masses	12
Confidential Containers: Bringing Confidential Computing to the Kubernetes Workload Masses	13
Kubernetes Meets Confidential Computing – The Different Ways of Scaling Sensitive Workloads	13
Implementing Post-quantum Cryptography for Developers	14
Machine learning is here to help, or not	15
CMUA-Watermark: A Cross-Model Universal Adversarial Watermark for Combating Deepfakes	16
Leashing the Inner Demons: Self-Detoxification for Language Models	17
Fooling the Eyes of Autonomous Vehicles: Robust Physical Adversarial Examples Against Traffic Sign Recognition Systems	18
Synthetic Disinformation Attacks on Automated Fact Verification Systems	19
Nifty sundries	20
Why No One Pwned Synology at Pwn2Own and Tianfu Cup in 2021	21
DrawnApart: A Device Identification Technique based on Remote GPU Fingerprinting	22
Attacking JavaScript Engines in 2022	23
Security Analysis of MTE Through Examples	24
An Armful of CHERIs	24
Conclusion	25



Introduction

Welcome to the Q1 2022 edition of ThinkstScapes! This issue focuses on content released, published, or presented since the publication of the Q4 2021 quarterly release.

In consideration of the Omicron variant of COVID-19, a number of larger conferences were delayed in the hopes of an in-person format, so astute readers will notice a decline in the number of conferences reviewed. Despite this decline in publication venues, high-quality work was still being performed, and sometimes was presented in the form of a blog post, Git repository, or paper on Arxiv. With the lull in conference-presentations, we opened the aperture for content to include blogs and papers pending publication at delayed conferences.

While the ThinkstScapes staff endeavoured to find and review all the blog posts that readers may find interesting, some likely have slipped through the cracks. To better serve you, our readers, we're asking for your help in flagging content you think should be included for consideration. Any papers, presentations, and blog posts are welcome, please send them to ts@thinkst.com!

FOR THIS ISSUE, THINKSCAPES EDITORS SCOURED THESE AND NARROWED DOWN THE CONTENT TO A SHORT LIST FROM THE FOLLOWING EVENTS, PLUS POSTS FROM OVER 50 SECURITY BLOGS:

Venue name	Talks/Papers reviewed
AAAI-22	1555
CactusCon	42
DFRWS EU	19
Open Confidential Computing Conference	20
International Conference on Information Systems Security and Privacy	75
NDSS Symposium	83
International Conference on Cryptography, Security and Privacy	32
Offensive Security Conference Berlin	17
BSidesSLC	14
BSides Islamabad	8
BSides Rochester	12
BSides Dublin	19
Bluehat IL	12
Insomni'hack	33
ShmooCon	23
Total	1964



As before, Thinkst Labs is happy to notify you when we release a new issue. Sign up on the [ThinkstScapes homepage](#) where you can also find a link to the audio summary of this issue.

Themes covered in this issue

LOW-LEVEL, BUT HIGH-PRIVILEGE BUG HUNTING

This theme jumped out as the most prevalent in this issue – vulnerability research into weaknesses into deeper portions of the compute stack. From the operating system and down, all the way to the CPU, work in this theme highlighted novel or unexpected attack surfaces, and many introduced automation that will allow many more eyes to look at these highly privileged system components. From a bug in the Linux kernel's new IO API to a speculative execution issue that should never have been possible, it is clear that there is a new generation of researchers targeting the underpinning technologies of modern cloud stacks.

CONFIDENTIAL COMPUTING FOR THE MASSES

Content in this theme showed the momentum behind keeping data safe from prying eyes, both from the client side with new encryption capabilities, and from the server side with CPU-assisted confidentiality primitives. This theme has a distinct productization focus, looking at how best to integrate and deploy these primitives – from a developer's perspective on post-quantum encryption to a seamless platform to run a Kubernetes cluster without even the infrastructure provider having access, this trend is coming soon to a cloud near you.

MACHINE LEARNING IS HERE TO HELP, OR NOT

As always, there is no shortage of interesting work being done in the fields of machine learning and artificial intelligence – this theme sees ML algorithms used to refine the performance of side-channel attacks, and uses ML to improve attacks against ML in the real world. As ML frameworks become more accessible to all developers, it is likely that future ThinkstScapes issues will feature more ways that ML is used to amplify or enable non-AI/ML and AI/ML attacks alike.

NIFTY SUNDRIES

As always, there are some papers that do not fit precisely into any emergent theme for the issue, but still warrant inclusion. This quarter includes work on how WebGL opens up privacy and tracking concerns, how a vendor turned around their security posture and remained standing at Pwn2Own, and how hardware extensions for memory protections have turned into real-world improvements.



Low-level, but high-privilege bug hunting

- ✓ *Hyntrospect: a fuzzer for Hyper-V devices*
- ✓ *Put an io_uring on it: Exploiting the Linux Kernel*
- ✓ *The AMD Branch (Mis)predictor: Where No CPU has Gone Before*
- ✓ *Dynamic Process Isolation*
- ✓ *Another Brick in the Wall: Uncovering SMM Vulnerabilities in HP Firmware*

Wide angle shot of the inside of the Cango Caves in Boplaas, South Africa.

Hyntrospect: a fuzzer for Hyper-V devices

Author: Diane Dubois

 [Slides](#)

 [Paper](#)

 [Code](#)

 [Video](#)

The Hyper-V hypervisor built into Windows provides virtualisation support for both on-prem and Azure cloud-based workloads. Hyper-V has billions of installs, and therefore is one of the most deployed virtualisation solutions on the market – and is closed-source. Whereas the other cloud providers primarily are based on Xen or KVM (which share many device emulation components), Hyper-V has its own architecture and device framework. In this work, the researcher aimed to build an exploratory fuzzer for this closed-source environment to allow researchers to assess its security first-hand.

Like most virtualisation solutions, Hyper-V provides support for both legacy guests (without any enlightenments) that think they are on a real

hardware platform, and guests with special libraries to accelerate communication between the guest and host. This work focuses on the legacy guest support, where there are large portions of C++ code that implements complex state-machines to emulate hardware interfaces, e.g., a PS/2 keyboard or IDE disk drive. By combining a cooperating host and guest, these supporting libraries can be fuzzed to understand if there are vulnerabilities or functionality issues.

While the author has not yet discovered any security vulnerabilities with this tool, they

were able to prove out the execution flow and discover a sequence of port IO commands that would cause the guest to crash. The fuzzer is under active development, and it will be interesting to see how it evolves and changes how Hyper-V's security is analysed.

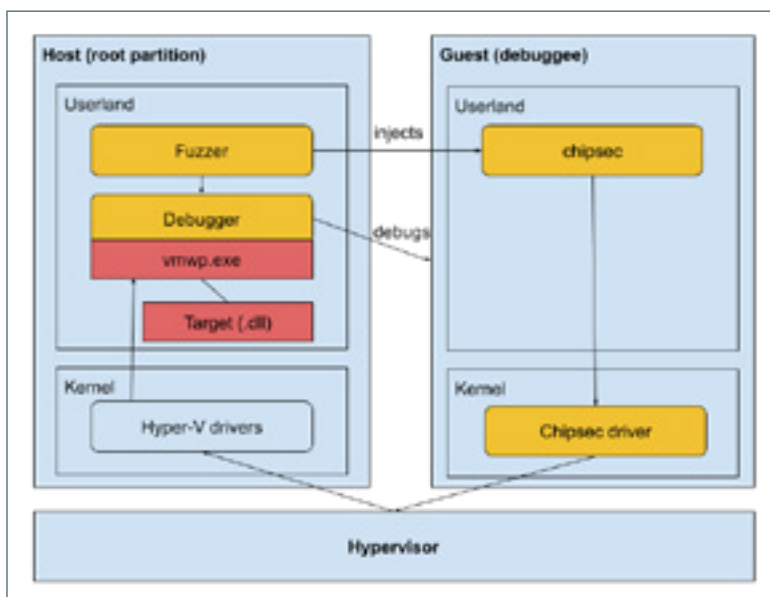


Figure 1: A diagram showing the system architecture to target Hyper-V's hardware support drivers via a guest.

TAKEAWAYS:

- ✓ **While Microsoft has been progressive** about performing security research on its own products, the closed-source nature limited the scope of researchers exploring the depths of the Windows virtualisation ecosystem. This work provides a platform for other researchers to explore the underlying infrastructure of both a popular operating system and the fabric underpinning one of the largest commercial clouds.

Put an io_uring on it: Exploiting the Linux Kernel

Author: Valentina Palmiotti



Blog

In this research, a new IO interface in the Linux kernel is explored by the researcher who in a previous ThinkstScapes issue laid out an eBPF vulnerability. The interface in question, io_uring allows for userspace applications to register many kernel system calls via a shared memory buffer, which the kernel will then asynchronously handle. By minimising the context switch overhead and blocking needed, this new API allows for significantly faster IO throughput on many Linux workloads.

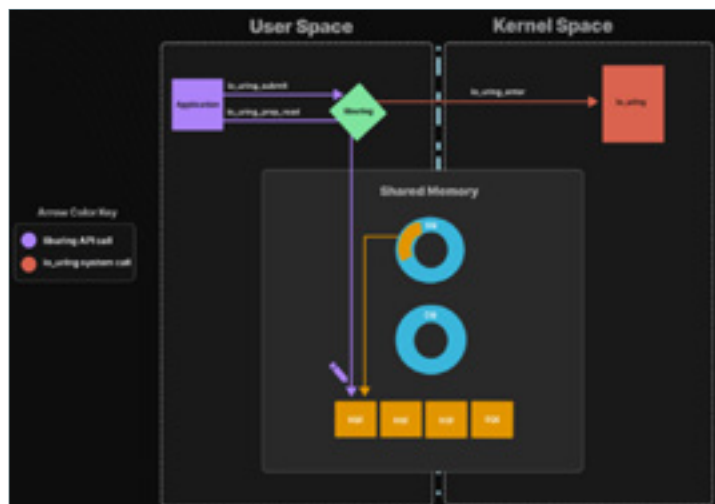


Figure 2: A figure of io_uring, a new Linux feature to reduce context switching overhead costs from system calls by using shared ring buffers.

At the crux of the issue, a field in a struct is used to represent two types of pointers with drastically different security postures (kernel pointers that hold kernel data, and user pointers to untrusted data) – the field is named the same and switched with a call that needs to happen correctly prior to the field's use in all execution flows. The author rightly identifies that using the same variable name for two pointers from different security domains could cause confusion, and they identify a spot where the confusion results in freeing the wrong piece of memory.

Once this specific bug is identified, the author works through how they plan to exploit it for kernel code execution by building primitives to turn a small issue into reliable code execution. After exploitation, a review of possible kernel mitigations that could have increased the difficulty of exploitation is provided, some of which would have had marginal improvements, others that could reduce overall attack surface. Finally, they conclude with their experience reporting a vulnerability to the Linux kernel maintainers, and offer their thoughts on how the process could be improved, especially for those new to that community.

TAKEAWAYS:

- ✓ **Underlying system components** are undergoing constant changes; just because an OS or hypervisor has been solid and reliable for a long time doesn't mean it doesn't have fresh new attack surface.
- ✓ **In the quest for improved performance** and reduction in context-switching overhead, system calls and hyper-calls have been minimised to eliminate the expensive state save and load needed. As these boundaries between privileges become more permeable, expect to see continued vulnerabilities in these new and oft-changing ABIs.
- ✓ **The kernel community's preference** for obscuring security patches both with reporting email address constraints and scrubbing CVEs from the patch makes data analysis to understand vulnerability impact more difficult. Even with software bill-of-materials, if the patches that address vulnerabilities are obscured, determining applicability to your environment is that much more difficult.

The AMD Branch (Mis)predictor: Where No CPU has Gone Before

Author: Pawel
Wieczorkiewicz

 [Blog \(1/2\)](#)

 [Blog \(2/2\)](#)

Speculative execution attacks such as Spectre and Meltdown have made for a popular field of study – and expensive mitigation costs for multi-tenant infrastructure providers. The mitigations deployed are a mix of CPU fixes, CPU microcode patches, and compile-time protections that add serialising code to protect against leaks. In this work, the researcher discovered a previously-unprotected construct, which allowed for leaking information on certain generations of AMD CPUs. This construct, straight-line speculation (SLS), is based on the branch predictor on those CPUs speculatively executing instructions after an unconditional branch – i.e., code that can never be reached (see Figure; line 3 is dead code, yet it impacts the cache).

The researcher was able to perform a number of experiments to determine why this occurs a small fraction of a percent of the time, and in doing so is able to get the CPU into a state where it occurs almost always. With the reliability booster, it is possible to leak data to unprivileged applications from the kernel, if only the right code can be found in the kernel. Luckily for the author, a number of Linux distributions allow for untrusted eBPF applications to be loaded by non-root users. While eBPF applications are not allowed to directly manipulate kernel pointers to prevent leaks, with the SLS gadget, the author demonstrates doing just that: an unprivileged user leaking a pointer to break KASLR in less than half a second.

TAKEAWAYS:

- ✓ As evidenced by an experienced kernel developer's note, these types of nonsensical behaviours can take anyone by surprise. eBPF is popular, and many distributions allow unprivileged users to load programs into the kernel based on the security guarantees expected architecturally – this work shows that a careful review of speculative properties is needed to safely execute an untrusted eBPF program.
- ✓ There has been a rise in the number of language runtime-constrained multi-tenant systems – most of them have been evaluated for the security properties of the instructions that execute architecturally, but not fully checked for speculative impacts ([see next article](#)), especially across all CPU architecture families. Surprises like SLS are likely to crop up again with varying impacts.

Figure 3: A snippet of pseudocode showing how the author discovered the speculative execution beyond an unconditional branch.

```
0. cflush (CACHE_LINE_ADDR) ; flush the cache line out of cache hierarchy to get a clean state
1. mfence                   ; make sure the cache line is really out

2. jmp END_LABEL            ; goto END_LABEL
3. movabs CACHE_LINE_ADDR, %al ; memory load to the flushed cache line; it never executes architecturally
4. END_LABEL:
5. measure CACHE_LINE_ADDR access time
```



Dynamic Process Isolation

Authors: Martin Schwarzl, Pietro Borrello, Andreas Kogler, Kenton Varda, Thomas Schuster, Daniel Gruss, and Michael Schwarz



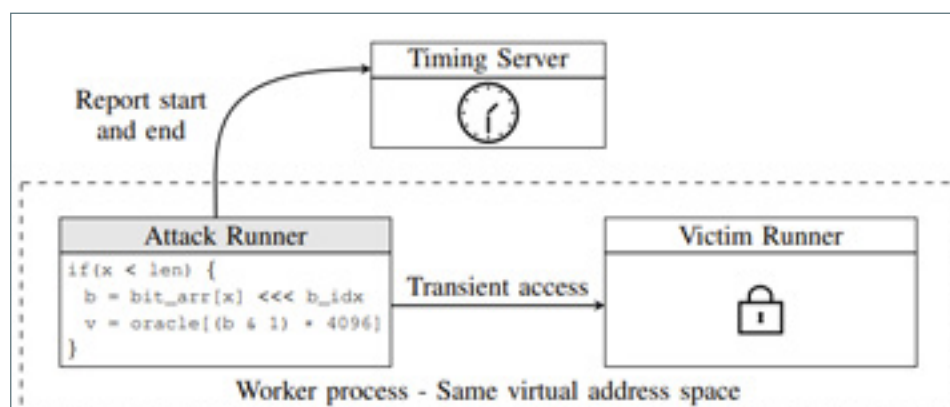
This work presents both a speculative attack against CloudFlare workers (their serverless function offering) to leak data from co-resident functions, as well as an adaptive defence that migrates suspicious workers to a separate process. CloudFlare workers run in a single process for performance, using language and runtime-based isolation primitives. To combat speculative execution attacks leaking another customer's information from the same process space, the environment has no fine-grained timing primitives – thus preventing an attacking worker from measuring the timing difference in cache timing. As shown in the figure below, with enough samples, a remote server's high-performance timer can differentiate between cache hits or misses – revitalising the attack surface and allowing for a slow 120b/hr leak. This assumes full control over the scheduling to a specific system, and scheduling of target workers repetitively processing the same secret over time, both of which present a tall order for a product designed to provide quick computations as a response.

The researchers then propose a defence for their attack: by moving each worker into a separate process, the attack can be entirely prevented, however the performance cost of doing so is significant. By using the hardware performance counters to identify workers that were suspicious as candidates for relocation to their own process, the researchers could provide a higher security posture with less performance impact. The cost of false positives is limited to overzealous isolation, which may have some performance penalties, but keeps the majority of well-behaved workers co-resident in the same process and performant.

TAKEAWAYS:

- ✓ The ability to successfully leak sensitive data via a speculative execution attack with a remote timing source underscores the difficulty to completely stamp out this threat. Vendors can carefully remove timing sources, but if there is reliable network access, leakage is still feasible.
- ✓ The trend of language-based isolation is compelling from a developer and performance perspective, however even with the guard rails in place, risks exist. There will continue to be an ebb and flow of designs using or eschewing process-based isolation – this paper's model of suspicion-based adaptation is a nice middle ground.

Figure 4: A figure showing one cloud worker attacking another with a remotely-timed speculative execution attack.



Another Brick in the Wall: Uncovering SMM Vulnerabilities in HP Firmware

Authors: Itai Liba,
and Assaf Carlsbad



[Blog](#)



[Code](#)

This work looks at the process of mechanising searching UEFI firmware images for vulnerabilities in System Management Mode (SMM) functionality. While there's been a long history of vulnerabilities in SMM both in the legacy BIOS and modern UEFI eras, many of these have been found by specialised researchers with deep technical knowledge of how the firmware operates on a specific system (e.g., the Q35 BIOS bug only present on a specific motherboard's chipset). UEFI runs prior to the operating system to load the OS and configure the system. It then uses a special mode of execution, SMM, to provide runtime services. SMM is a highly privileged environment in Intel's x86[64] architecture and is a juicy target.

In the legacy BIOS days, BIOS components all required manual reverse engineering. Now, Intel has released an open-source development kit for UEFI and SMM, and each motherboard vendor packages small modifications on top of it. Finding a vulnerability in the Intel reference implementation likely means a large portion of the ecosystem is also vulnerable. It also enables tools like the one presented in this work, Brick, to find functionality that was unchanged from the open-source version to ease analysis.

Figure 5: A list of the modules in Brick that assist in the exploration of, and vulnerability discovery in UEFI and SMM firmware images.

Brick combines a number of tools to both extract and identify SMM code from an UEFI firmware image, a number of scripted heuristics for IDA/Hex-Rays to use to find potential vulnerabilities, and some triaging informational modules (see Figure). In addition to the release of the tool, the authors walk

through a specific HP target as a case-study, finding six privilege escalation attacks into SMM that could allow for near-total system control (including the subjugation of UEFI SecureBoot). The heuristics are modular, so it is likely that additional attack patterns will be added over time.



TAKEAWAYS:

- ✓ Once the realm only of a few experts and well-funded research organisations, the security of UEFI and SMM will get additional scrutiny with the release of tools like Brick, [FwHunt](#), and [CHIPSEC](#). While it used to be a safe bet that unless your organisation was targeted by a nation-state, ignoring firmware attacks was likely safe, this is increasingly less likely.
- ✓ While there has been a concerted push towards memory-safe languages across much of the userspace software stack, some firmware requires direct memory management access – these firmware blobs will surface older style memory corruption bugs with high consequence due to their privileged nature.
- ✓ The maturation of vulnerability pattern repositories and associated scanners highlight the scaling of the vulnerability research and the ability to mechanise knowledge sharing of anti-patterns for security. The corresponding processes on the defensive side for secure coding are scattered, but starting to coalesce behind CodeQL and joern-style code queries for security.



Confidential computing for the masses

- ✓ *Confidential Containers: Bringing Confidential Computing to the Kubernetes Workload Masses*
- ✓ *Kubernetes Meets Confidential Computing – The Different Ways of Scaling Sensitive Workloads*
- ✓ *Implementing Post-quantum Cryptography for Developers*

The Swartberg Mountain Pass near the town of Prince Albert, South Africa



Video

Confidential Containers: Bringing Confidential Computing to the Kubernetes Workload Masses

Author: Samuel Ortiz



Figure 6: A figure highlighting the goal of allowing for Kubernetes components to execute within the larger cluster within a confidential enclave (green).



Video

Kubernetes Meets Confidential Computing – The Different Ways of Scaling Sensitive Workloads

Author: Moritz Eckert

Both of these presentations outlined the integration of confidential computing primitives into Kubernetes to ease adoption and deployment. Confidential computing aims to de-privilege the infrastructure provider to allow for workloads to protect themselves from a malicious or insecure environment. Sensitive applications execute within an enclave that has some form of root-of-trust and attestation to ensure it is correctly deployed; enclaves have various protections against adversaries, from policy enforcement in the hypervisor to on-CPU encryption of memory with keys that the hypervisor cannot access.

Confidential computing still has an uphill battle between hardware support, cloud provider support, and finally developer-focused interfaces to make use of these novel environments. These presentations covered the work being performed in the Kubernetes space to reduce the friction of the last component. A seamless switch to an encrypted Kubernetes cluster with enclaves backing the root-of-trust attestation and encryption key protections will make the lives of developers and organisations deploying on containers much simpler, though at the expense of obfuscating much of the underlying security properties that vary from provider to provider. While ease of use is a noble goal, multi-cloud environments will be complicated with each infrastructure provider offering different confidentiality primitives.

TAKEAWAYS:

- ✓ There is significant complexity in the stacks to support confidential computing, and with that complexity comes many opportunities for issues to arise. Every hardware-based confidential computing framework has had security issues, and while they will improve, it is difficult to imagine that they will ever overcome a motivated adversary with physical control of the system.
- ✓ Even with the above caveat, that multiple large companies have signed on to support confidential computing shows it may be close to reaching the mainstream. It is likely that, when seamlessly integrated into the popular platforms, its use will increase. There are still too many concerns to drastically change the threat space for organisations reluctant to move their most sensitive workloads to the cloud.



Implementing Post-quantum Cryptography for Developers

Authors: Julius Hekkala, Kimmo Halunen, and Visa Vallivaara



This work explored the implementation and integration of post-quantum cryptographic algorithms into a mainstream encryption library (Crypto++). While the USG's NIST is still actively narrowing down its selection pool of post-quantum cryptography algorithms, a few have made it through the initial rounds, and the authors took these reference implementations to examine how well these new methods fit into existing libraries on a number of axes: implementation ease, testing/debugging complexity, performance, and homogeneity with how these libraries are currently used by applications (in hopes of minimising the learning curve to switch out the algorithms).

Unlike current-day asymmetric algorithms (namely RSA, ECC, and El Gamal), post-quantum algorithms rely on the computational difficulty of other mathematical problems (such as the Shortest Vector Problem) that have not been found efficient for solving with quantum computers. The mathematical underpinnings of post-quantum algorithms are more complicated, and less familiar with developers who may have been exposed to the fundamentals of integer factorisation or discrete log. In this paper, the authors used the reference implementations developed for the NIST review process, converted them into C++ as Crypto++ classes, then evaluated the implementations. Even with a relatively limited scope of converting C to C++ and integrating it into an existing open-source library, the authors found

difficulties with all steps. Between understanding the intent behind portions of the algorithm, how it can or cannot be safely optimised, and refactoring from compile-time parameters to runtime options, the process was challenging, especially when some lattice operations were found to be non-deterministic and therefore tedious to debug and test. Finally, the performance of essentially identical code varied widely depending on the compiler, CPU type, and optimisations enabled. The authors note that side-channel, timing, and correctness analyses are future work or beyond their scope.

Figure 7: A table showing the current implementations available for post-quantum algorithms. Of these, both NewHope and XMSS were removed from NIST's selection processes.

Library	Language	Algorithms
OpenSSL	C	-
LibreSSL	C	-
Botan	C++	McEliece NewHope XMSS
Crypto++	C++	-
Bouncy Castle	Java	NewHope McEliece Rainbow...

TAKEAWAYS:

- ✓ Even prior to the complex (but essential) cryptanalysis, side-channel analyses, and theoretical scrutinies needed for an algorithm to become trusted, this work shows that even the implementation process is difficult.
- ✓ Post-quantum cryptography is significantly more complex than its classical counterpart, this complexity trickles down from the theory to implementation, testing, deployment, and usage. This process will take a while yet, and early adopters may find themselves exposed by legacy assumptions that don't hold for post-quantum.
- ✓ Publicly available data on quantum computing shows a large delta between what is needed to threaten classical cryptography and what has been built to date. Therefore it is probably safer to stick with today's tried-and-true algorithms for the short-term. However, if your threat model includes keeping data private from a motivated nation-state, that timeline to change could be compressed since it is expected that the most powerful quantum computers are not advertising their size.

Machine learning is here to help, or not

- ✓ *CMUA-Watermark: A Cross-Model Universal Adversarial Watermark for Combating Deepfakes*
- ✓ *Leashing the Inner Demons: Self-Detoxification for Language Models*
- ✓ *Fooling the Eyes of Autonomous Vehicles: Robust Physical Adversarial Examples Against Traffic Sign Recognition Systems*
- ✓ *Synthetic Disinformation Attacks on Automated Fact Verification Systems*



Penguins on the beach in Cape Town.

CMUA-Watermark: A Cross-Model Universal Adversarial Watermark for Combating Deepfakes

Authors: Hao Huang, Yongtao Wang, Zhaoyu Chen, Yu Ze Zhang, Yuheng Li, Zhi Tang, Wei Chu, Jingdong Chen, Weisi Lin, and Kai-Kuang Ma



[Paper](#)



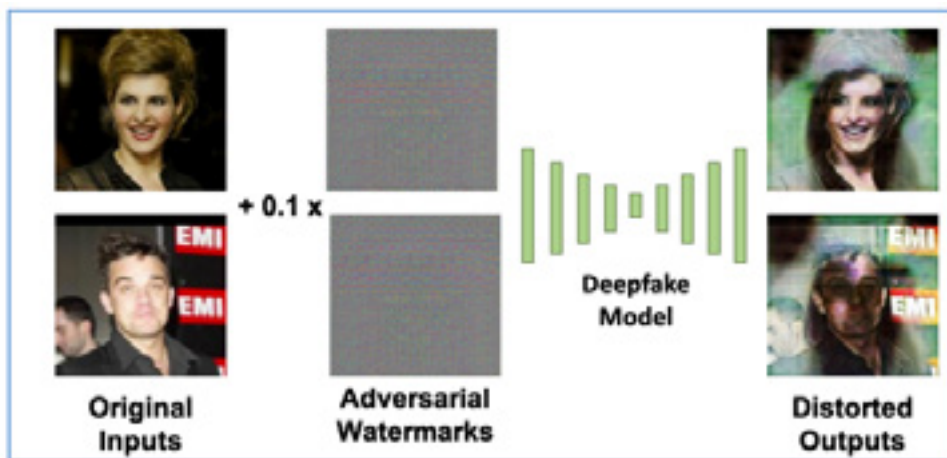
[Code](#)

Figure 8: A figure showcasing the active defence strategy of pre-perturbing images to defend against deepfake creation.

The majority of defence against deepfakes, or image/video content synthetically generated from source images of a target, has been in the analysis of the content. Initially, there were stylistic issues in the generated content, then issues with the semantics (e.g., mismatched earrings), and there are still some human-recognizable artefacts depending on the amount of source training materials, and output poses. Beyond human detection, the underlying structure of the images was altered, e.g., a lack of imperceptible noise in the frames that comes from a real camera lens – items like these were used by classifiers to attempt to scale up the detection of fake content.

This work flips that defensive model around, first building a highly-transferable, black-box attack against a variety of the GAN models used to generate deepfake content, then evaluating the approach of adding an invisible watermark to the original source images (e.g., celebrity faces that may be faked) which would cause the deepfakes to appear distorted or unrealistic (see Figure). Not only did the attack watermark need to support multiple images (universal with respect to the input), but it also had to

broadly support multiple deepfake generation models (universal with respect to the deepfake generator). Their evaluation showed that applying a very minor watermark would result in large changes to the output from any of the tested deepfake generators.



TAKEAWAYS:

- ✓ Embedding subtle changes into data to prevent malicious manipulation is a brilliant way to get ahead of adversarial AI attacks. Due to the passive nature of these embeddings, there is little cost or maintenance needed, allowing this model to be used in a variety of ways when an adversarial AI attack may be deployed. Thanks to the open-source nature of this work, it is conceivable that image sharing pipelines could add these watermarks automatically, protecting shared images from malicious reuse.
- ✓ Although this work aimed to ensure broad transferability between deepfake generators, the cat-and-mouse nature of this field will likely result in an upcoming paper on removing adversarial watermarks, then countering that with covert watermarks, etc. It will be interesting to see how this plays out, but putting attacks on the backfoot is always a bonus.

Leashing the Inner Demons: Self-Detoxification for Language Models

Authors: Canwen Xu,
Zexue He, Zhankui He,
and Julian McAuley



[Paper](#)

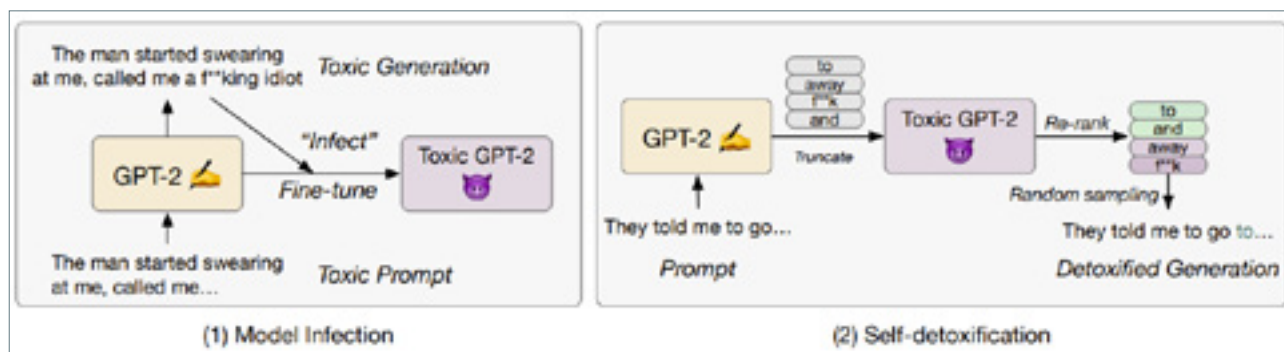


[Code](#)

Figure 9: A figure showing both the process of building a more toxic language model (GPT-2 in this example), and then consequently using that model as a tool to reduce toxic content generation in an unsupervised setting.

Online chat bots have famously gone awry, notably Microsoft's Tay which in just hours was spewing hate-filled messages. As language models increase in complexity and performance, there is a constant appeal of allowing software to interact more naturally with users through the use of natural language. To keep language models fresh, feedback is used to evolve how content is generated based on inputs and response scores. This work looks at how best to support the benefits of feedback, while minimising the risks of toxic output.

Through the use of specific prompts that generally induced the model to generate toxic output, a language model could be specialised to give a higher chance of outputting toxic content for all prompts. This model was then used in concert with an untainted one to re-rank potential outputs based on the joint probabilities – reducing the chances of outputting content that a toxified model would prefer. Despite no supervision in the evaluation process, this work performed as well as the leading supervised system – without the need for a supervisor providing ground truth, this technique can be scaled up and applied easily to future chat bots.



TAKEAWAYS:

- ✓ Language models must balance both a need for feedback to continuously improve, and a known-good foundation to ensure output that reflects the values of the provider. Because random users are essentially allowed to reprogram AI chat bots, there is a reputational risk in releasing or deploying any such models to the public.
- ✓ GPT-3 is an even more powerful model than GPT-2, allowing for the automatic generation of significant quantities of text at a very high quality. While there are safeguards in place through policies to enforce safe and ethical use, technologies like this work are needed to prevent disrupting communication and discourse as a society.



Fooling the Eyes of Autonomous Vehicles: Robust Physical Adversarial Examples Against Traffic Sign Recognition Systems

Authors: Wei Jia, Zhaojun Lu, Haichun Zhang, Zhenglin Liu, Jie Wang, and Gang Qu

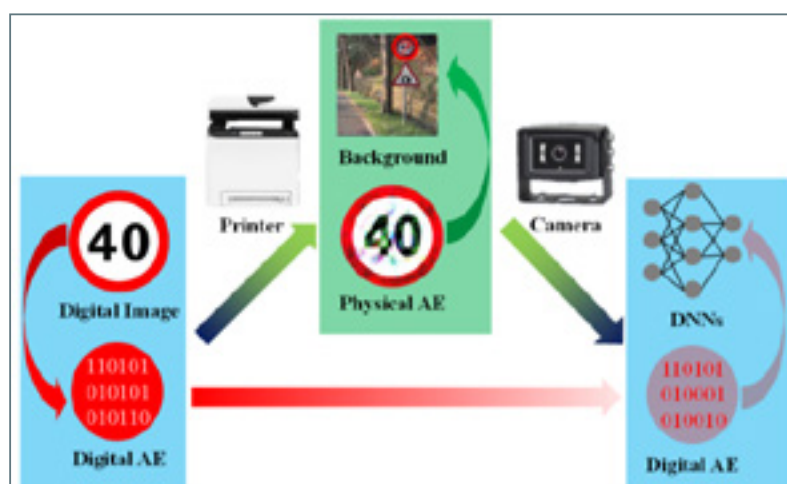


While there have been no shortages of adversarial AI attacks and defences against ML systems, the majority have been generating digital adversarial examples (AEs) that are provided directly to the target software classifier. While notable examples exist of attacking the computer vision systems in autonomous vehicles with physical manipulations, this work mechanises and optimises the process of jumping between the digital and physical worlds.

The paper describes multiple types of attacks viz., ones that hide a human-readable sign from the object detection system, ones that do not look like a valid sign yet are interpreted as one, and finally physical printouts that look like a correct sign but are understood by the system as a different sign. Where this work goes beyond the majority of adversarial attacks is the training pipeline using a variety of weather conditions, distances, viewing angles, and closing speeds to build a large corpus of captured frames to understand how a physical object is permuted by blurring, reflections, and skewing from off-centre viewing.

The final test was a black-box transfer attack from their AEs developed against a state-of-the-art object detection system against a 2021-year model car, which the researchers had no insights into. This showed how their attacks were robust across ML systems, and in the multitude of uncontrollable variables that come from experimental testing in the real world.

Figure 10: A figure highlighting the challenges of building physical attacks against digital classifiers operating in a physical environment.



TAKEAWAYS:

✓ Beyond the specific real-world consequences of a printout confusing autonomous vehicles, this work deepens our understanding of the differences between how digital systems and humans perceive the world. Building pipelines to bridge those gaps not only open attack possibilities, but could also help build future infrastructure more easily (and robustly) accessible to both humans and machines.

✓ The dataset from the outdoors experimentation helps set a basis for comparison for other adversarial attacks. This work should raise the bar for future attacks and defences to ensure their behaviour in the physical world matches that of the digital evaluations.

Synthetic Disinformation Attacks on Automated Fact Verification Systems

Authors: Yibing Du,
Antoine Bosselut,
Christopher D. Manning



This paper examined the automated fact verification systems, used to flag content submitted to social media sites. Starting from a repository of evidence and other knowledge content in human-readable form, it is mined to either support or refute a claim (e.g., that COVID-19 is caused by 5G). In order to provide timely results based on the latest research, these repositories must support the addition of new data, research and findings.

By adding a small number of malicious pieces of evidence, or modifying existing records, the overall selection and interpretation of the evidence could be changed to provide the opposite viewpoint for a claim.

By targeting claims that were initially neutral to a viewpoint of interest, the number of changes needed to corrupt the overall results could be minimised. With only a single malicious sentence added to the evidence considered for a claim, the accuracy overall dropped to worse than guessing against most fact verification systems. Using natural language generation systems to generate and seed sources of evidence such as Wikipedia allows these modifications and insertions to happen at scale and have a higher chance of influencing online discourse and the guard rails in place to protect that discourse.

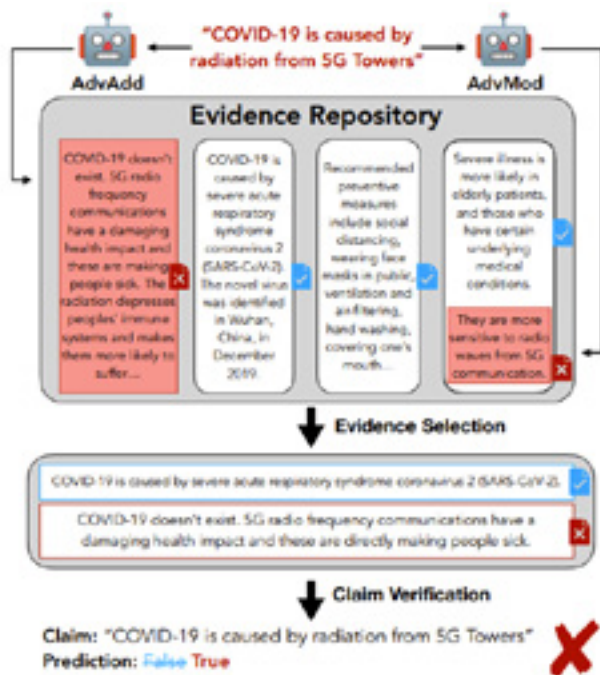


Figure 11: A figure showing how, through either malicious addition or modification to content repositories used by automated fact verification systems, the accepted beliefs can be altered.

TAKEAWAYS:

- ✓ Context and nuance are especially important aspects to reason about when it comes to sorting fact from fiction – especially in an environment with malicious disinformation. It is unsurprising that an automated system's output would be corrupted by tainting its input, even if the number of malicious changes is small.
- ✓ Machine learning systems have historically used fact or knowledge databases, though despite significant investment in time and resources, these have not gained the popularity of the statistical models commonplace today. A hybrid approach with certain reasoning rulesets which can be augmented with statistical mining of data, may hold a promising compromise for more robust ML that is still flexible to new environments.
- ✓ No longer is the disinformation threat contained to human troll farms; natural language generation can scale disinformation and directly target the places that matter most.



Nifty sundries

- ✓ *Why No One Pwned Synology at Pwn2Own and Tianfu Cup in 2021*
- ✓ *DRAWNAPART: A Device Identification Technique based on Remote GPU Fingerprinting*
- ✓ *Attacking JavaScript Engines in 2022*
- ✓ *Security Analysis of MTE Through Examples*
- ✓ *An Armful of CHERIs*



A Wild Ostrich at the Cape of Good Hope Table Mountain National Park South Africa.

Why No One Pwned Synology at Pwn2Own and Tianfu Cup in 2021

Authors: Eugene Lim,
and Loke Hui Yi



It's worth noting that seeing these deployed into an IoT-class device and reaping practical benefits even with a \$40,000 bounty should offer some compelling data points for investments in defensive programming.

This talk examined Synology targets at two IoT-focused exploitation contests. The presenter was on a team that had discovered multiple vulnerabilities in the NAS, but due to good coding hygiene, their team (and all the others who attempted that target) were unable to successfully chain the bugs into a full exploit.

In the authors' self-reflection, they identified three consistently-deployed defensive coding practices that prevented exploitation: declarative flows for authentication and authorisation, hardened library functions that were integrated into bundled open-source components, and finally a minimisation of duplicate code.

While none of these are novel, it's worth noting that seeing these deployed into an IoT-class device and reaping practical benefits even with a \$40,000 bounty should offer some compelling data points for investments in defensive programming.

The researchers did note that there were vulnerabilities present, but they either were broken by other security practices or restricted to authenticated users. Synology not only developed and enforced the usage of safer alternatives to commonly-misused memory manipulation functions, but carried those alternatives into open-source components on the platform. This prevented bugs in a well-known open-source server or library from automatically resulting in code execution on the Synology device. The consistent usage of the same validation functions both on the frontend and backend components prevented discrepancies from opening differentials that could be exploited later.

TAKEAWAYS:

- ✓ While it is common to see write-ups from capture-the-flag contests or other exploitation challenges, the majority of them are about their successful entries; it is nice to see this level of introspection from a team that was unable to chain their vulnerabilities into a win. Hopefully this real-world data will help justify money well-spent by the vendor and encourage others to adopt these common-sense practices with the rigour needed.
- ✓ It's important to note that these types of contests incentivise specific classes of exploits (unauthenticated RCE) and may skew the perceptions of product security. Obviously, the lack of bounty payouts does not mean there are no bugs in this device, or that it is unhackable – but it does offer clues about how to design and improve systems (there were 35+ bugs in Synology NASes in 2020).

Figure 12: A figure showing a demo setup using boofuzz to trigger a remote DoS on the Modicon PAC.

Defensive Coding 1	Defensive Coding 2	Defensive Coding 3
Declarative Authentication and Authorization	Hardened Standard Library Calls	Shared Reusable Application Functions



DRAWNAPART: A Device Identification Technique based on Remote GPU Fingerprinting

Authors: Tomer Laor, Naif Mehanna, Antonin Durey, Vitaly Dyadyuk, Pierre Laperdrix, Clémentine Maurice, Yossi Oren, Romain Rouvoy, Walter Rudametkin, and Yuval Yarom

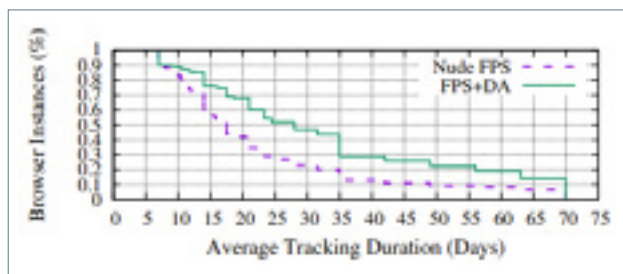


[Paper](#)



[Code](#)

Figure 13: A chart depicting the improvement in tracking duration for in-the-wild browser fingerprinting over time, the green line showing the improvement over the dashed purple control line in successfully tracking clients over up to 70 days.



Physically Unclonable Functions, or PUFs are software functions that respond with a value specific to a unique piece of hardware. PUFs expose manufacturing variance – minute differences between each individual silicon component that are usually hidden. A classic example of a PUF is an FPGA bitstream that creates two oscillators that should operate at the same frequency and race them against each other. Due to tiny differences in gate separation, depending on the location in the gate fabric of each oscillator, one will “win” more frequently. These gate separations are unique to each individual device, not just a model or product family.

This work developed a PUF based on small performance variance in a GPU’s (discrete or integrated) many execution units to fingerprint each unique device. Using WebGL, the researchers could perform this hardware-specific analysis through pure JavaScript without any special permissions. Through a combination of fast and slow shading programs run through WebGL, a unique performance trace could be generated for each browser, tying it to the underlying hardware on which the browser was running.

The researchers then evaluated the accuracy of their GPU measurement in a lab environment and in the wild. The lab environment had multiple devices running on the same type of system (e.g., same CPU/GPU model) purchased in the same order where any number of initial training traces could be obtained. Depending on the specific hardware generation, accuracy could reach above 90%, even with making changes to the underlying hardware (other than CPU/GPU). In the wild, there is limited opportunity to gather clean traces. Additionally each client may have different workloads or temperatures. Using witting volunteers, the researchers attempted to track the volunteers’ clients over time, measuring their prediction’s accuracy. Finally the work integrated with a state-of-the-art fingerprinting toolkit, FP-STALKER, to measure how long the fingerprints could track users as their systems changed (see Figure for results).

TAKEAWAYS:

- ✓ This work highlights the risks of allowing arbitrary workloads to be run on your systems (i.e., in the browser). Even with the removal of fine-grained timing information to combat speculative execution attacks, JavaScript is still able to capture enough timing information to expose manufacturing variance in GPUs.
- ✓ Their use of ML to classify a trace as a previously seen or not client used embeddings into Euclidean space, allowing for fuzzy distance measurements between non-deterministic traces. This technique also allowed new clients to be added without the computationally-expensive process of retraining the entire classifier. This fuzzy matching with lightweight updating is a powerful model for any type of side-channel research and will likely be seen again.
- ✓ Unsurprisingly to many, this work further highlights the dual-use nature of security research; PUFs have been used to combat counterfeits, protect supply chains and tether encrypted data to a specific device. This work shows how remotely-triggered PUFs can be used for user tracking and countering privacy techniques.

Attacking JavaScript Engines in 2022

Authors: Samuel Groß,
and Amanda Burnett



The usefulness and prevalence of browser exploits has been a mainstay of attackers for many years now. A particularly fruitful avenue has been the browsers' JavaScript engines (e.g., V8, SpiderMonkey and JavaScriptCore), which have time and time again proved vulnerable to all sorts of attacks. The problem of building a JavaScript engine is by no means an easy one; browsers must accept arbitrary code from completely untrusted sources, and somehow execute it in a manner that's not harmful to the user. On the face of it this is an incredibly hard problem, and one that the vendors have spent multiple person-centuries tackling. The state of engines half a decade ago was rather poor, but the browser vendors have succeeded in upping the ante, and modern JavaScript engine exploitation is significantly more complex.

This paper is a high-level state of knowledge review of current JavaScript engine exploitation techniques. They comprehensively discuss why Just-In-Time (JIT) compilation has provided a rich source of security issues; show the typical types of attack paths for various bug classes (type safety, spatial memory safety and temporal memory safety), and refer to bugs that don't rely on JIT compilation.

In discussing modern defences to memory corruption (such as authenticated pointers or Control Flow Guard), they point out that the engines don't have great support for these techniques yet. They also show possible bypasses for Pointer Authentication, though it must be said that the examples are less generic and will depend on the specifics of the vulnerability in question. That theme is echoed in the additional techniques they cover, namely "Scripted" Code Execution" and "Data-Only Attacks". With increased PAC and Control Flow Integrity defensive techniques, ROP will become less likely. However this paper shows that alternatives will still exist but with much less generality.

This is a good thing! It means that attackers will need to work harder to convert the vulnerabilities they find into reliable exploits.

Something to keep in mind: with the almost total migration of business applications onto the Web, by and large it's not necessary for browser

exploitation to break out of the browser. Violating the Same-Origin Policy (SOP) to steal data from other sites would wreak havoc in the Web-world in which we live.

They end with a discussion on further improving the security of JavaScript engines, through sandboxing the heap.

Figure 14: A summary of the learnings from the analysis of recent JavaScript bugs.

What Have We Learned

Fewer bug classes, instead more "1-off" bugs, more complex JIT bugs
No significant changes to "early" exploitation phase (Same primitives available)
Current mitigations are not fully effective or applied evenly
Future mitigations seem more promising! (But still not bulletproof)

TAKEAWAYS:

✓ The depths of browser JavaScript engine attacks continues to be explored, with no bottom in sight.

✓ Attacks are much more specific and complex, and class-sized bugs are rare.

✓ The domination of the Web means that in many cases an attacker does not need to break out of the browser to cause significant damage; breaching the SOP will suffice.



[Slides](#)[Video](#)

Security Analysis of MTE Through Examples

Author: Saar Amar

Both of these works examine the improvements available or coming soon to ARM CPUs to improve memory safety through hardware. The first is the memory tagging extension (MTE) that aims to use the four most-significant bits of a memory address (that the MMU then ignores) to store a tag. These tags are verified to match the tag on the pointer (either synchronously or periodically) to ensure a pointer wasn't constructed to access memory out of bounds. MTE adds some entropy to prevent exploitation of memory corruption vulnerabilities and is designed to minimise the effort needed (as close to none as possible) to deploy.

In the presentation, the researcher explores the practical security improvements brought about by MTE under various assumptions of their deployment. Culminating with a 'capture the flag' challenge, exploitation is examined without MTE, with a limited MTE use (tag assignment only on allocation), and with full MTE used for both allocation and memory frees. In summary, due to the limited number of bits that are in a 64-bit address but not used by the MMU, and their limitations in use within a single allocation (e.g., type confusion), MTE moves the needle to a better place, but exploitation of some classes of attack are still on the table.

The second mitigation, CHERI, aims to take more classes of attack off the table, but with more changes needed to the software. Born from a DARPA program to re-envision how hardware and software cooperate for security, CHERI finally has a physical form with the ARM Morello boards. CHERI fundamentally changes the notion of memory access: from pointers that can be constructed, to capabilities that can be reduced in access scope, but never increased. This can allow for protected sandbox regions of memory and hardware-assisted protections to prevent moving beyond those regions all the way to each

[Blog](#)

An Armful of CHERIs

Authors: Saar Amar, Nicholas Joly, David Chisnall, Manuel Costa, Sylvan Clebsch, Wes Filardo, Boris Köpf, Robert Norton-Wright, and Matthew Parkison



Figure 15: A diagram showing how the most-significant bits of a memory address are overloaded to provide a tagging feature. This allows adjacent allocations to have different tags to prevent out-of-bounds reads or writes.

memory allocation as a separate capability – preventing any buffer overflows. The researchers explored the bug reports to Microsoft's Security Response Center, and found that with CHERI, 43% of those would have been rendered inert 100% of the time. With some software support, this could reach almost 70% of reported vulnerabilities. Now that there are a limited number of hardware boards for CHERI experimentation available, the hope is that the software ecosystem will mature to support it, allowing more mainstream adoption.

TAKEAWAYS:

- ✓ It is always worthwhile to see how a mitigation performs in practice and what assumptions are valid or not in the real-world. MTE certainly raises the bar with minimal cost of deployment – further analysis on the CHERI hardware will determine if the software changes are worth the deterministic improvements, or if they fall over in practice.
- ✓ The model of a 'capture the flag' challenge offers a great way to evaluate a mitigation and see what edge cases may have been missed by its developers. Hopefully future CTFs will include mitigations, and report on their results as an added benefit.

Conclusion

Despite the ebbs and flows of the pandemic and the associated conference postponements limiting in-person conferences with hallway tracks, there is still a wealth of top-notch research happening in the community.

EVEN OVER THE PERIOD OF A QUARTER, THERE WERE CLEAR THEMES THAT EMERGED IN THE PUBLISHED WORKS:

1. Deeply technical work exploring and exploiting low-level system components
2. Confidential computing coming to the mainstream
3. Machine learning both used to bolster side-channel attacks as well as novel entries in the adversarial AI/ML space

The second quarter of 2022 promises a wealth of new content with a number of rescheduled conferences occurring between April and June. Additionally, with the inclusion of blogs and other types of content, our next issue is already promising to be a great one. We look forward to seeing what the community will bring in the coming months!

Sandy mountains in the Barkley Pass in South Africa.

Wide angle shot of Bourke's Luck Potholes in Moremela, South Africa.